

Гра з плиткою

Барчін та Харос грають у гру на сітці з $2N \times 2M$ одиничних квадратних клітинок. Рядки пронумеровані зверху вниз від 0 до $2N - 1$, а стовпці пронумеровані від 0 до $2M - 1$ зліва направо. Для $0 \leq i < 2N$ та $0 \leq j < 2M$, позначаємо клітинку в рядку i та стовпці j через (i, j) .

Барчін дає Харос $N \cdot M$ **блоків**, один за одним. Кожен блок — це квадрат 2×2 що складається з чотирьох 1×1 **плиток**. Барчін зафарбував кожную плитку блоку в чорний або білий колір, гарантуючи, що **принаймні одна плитка біла**.

Харос повинна розмістити кожен блок на сітці **відразу після його отримання**, не знаючи, які блоки вона отримає пізніше. Блоки не можна обертати. Кожен блок має бути повністю розміщений всередині сітки, покриваючи рівно чотири клітинки сітки. Крім того, верхній ліва плитка кожного блоку має покривати клітинку, координати рядка та стовпця якої парні. Кожна клітинка сітки може бути покрита щонайбільше одним блоком.

Барчін виграє гру, якщо після розміщення будь-якого блоку існує квадрат клітинок розміром 2×2 , покритий чотирма чорними плитками. Формально, якщо клітинки (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ покриті чорними плитками для деяких $0 \leq a < 2N - 1$ та $0 \leq b < 2M - 1$, то Барчін виграє. Індeksi a та b не обов'язково мають бути парними.

Харос виграє, якщо вона розмістить усі $N \cdot M$ блоків, і Барчін жодного разу не виграє. Зверніть увагу, що розміщення $N \cdot M$ блоків повністю покриє сітку.

Ваше завдання — впровадити стратегію, щоб Харос виграла гру. Можна довести, що за заданих обмежень Харос завжди може розмістити блоки так, щоб гарантувати перемогу, незалежно від кольору блоків, які вона отримає пізніше.

Деталі реалізації

Вам потрібно реалізувати дві процедури:

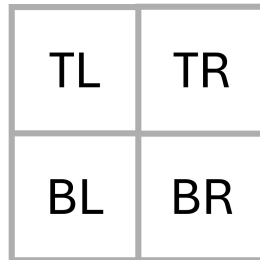
```
void init(int N, int M)
```

- N : половина кількості рядків у сітці.
- M : половина кількості стовпців у сітці.

- Процедура викликається рівно один раз для кожного тестового випадку, на початку виконання вашої програми.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : кольори верхньої лівої, верхньої правої, нижньої лівої та нижньої правої плиток поточного блоку відповідно, як показано на рисунку нижче. Кожне значення дорівнює або 0 (білий), або 1 (чорний).
- Ця процедура викликається рівно $N \cdot M$ разів у кожному тестовому випадку, після початкового виклику `init`.



Ця процедура має повернути пару цілих чисел (i, j) , де i – координата рядка, а j – координата стовпця клітинки, де слід розмістити верхню ліву плитку цього блоку. Як i , так і j мають бути парними, а область 2×2 покрита блоком, не повинна перекривати жодний попередньо розміщений блок.

Якщо `receive_block` коли-небудь поверне пару, яка не відповідає цим вимогам, або якщо після розміщення блоку квадрат клітинок розміром 2×2 буде повністю покритий чорними плитками, програма оцінювання негайно завершить вашу програму, а вердикт для тестового випадку буде таким: `Output isn't correct`.

Поведінка градера **не адаптивна**. Це означає, що послідовність блоків, які Барчін передає Харос, фіксується до виклику `init`.

Обмеження

Для кожного блоку нехай S буде кількістю чорних плиток серед його чотирьох. Тобто, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ для кожного блоку.

Підзадачі

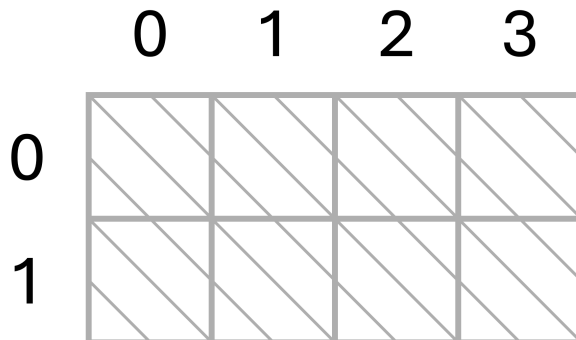
Підзадача	Бали	Додаткові обмеження
1	6	$S = 1$ для кожного блоку і $N = 2$.
2	16	$S = 3$ для кожного блоку. $N = M$, N парне, і кожне з чотирьох можливих розфарбовувань блоків з'являється рівно $\frac{N^2}{4}$ разів.
3	10	$S = 1$ для кожного блоку.
4	29	$S \leq 2$ для кожного блоку.
5	39	Без додаткових обмежень.

Приклад

Розглянемо гру з $N = 1$ та $M = 2$, тобто сітка має 2 рядки та 4 стовпці. Спочатку градер викличе:

```
init(1, 2)
```

Спочатку всі клітинки порожні. Сітка виглядає так:



Потрібно розмістити $N \cdot M = 2$ блоків. Припустимо, що Барчін дав блок із трьома чорними плитками та однією білою плиткою у верхньому правому куті. Градер викликає:

```
receive_block(1, 0, 1, 1)
```

Харос вирішує розмістити цей блок у лівій частині сітки, повертаючи $(0, 0)$.

Тепер сітка виглядає так:

	0	1	2	3
0				
1				

Потім Барчін дає ще один блок з трьома чорними плитками та однією білою плиткою у верхньому лівому куті:

```
receive_block(0, 1, 1, 1)
```

Єдина клітинка з парним рядком і парним стовпцем, яка може служити верхнім лівим кутом блоку 2×2 це $(0, 2)$, тому Харос повертає $(0, 2)$. Сітка в результаті має такий вигляд:

	0	1	2	3
0				
1				

Жодна клітинка розміром 2×2 не є повністю покритою чорними плитками, тому Харос успішно розмістила усі блоки, і Барчін жодного разу не виграв. Харос виграє гру.

Приклад градера

Формат вхідних даних:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Формат вихідних даних:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Тут $R[k]$ та $C[k]$ це пара цілих чисел, що повертаються k -м викликом `receive_block`.